



Programação Avançada 2025-26

[12] Padrões de Software | Introdução e Iterator Pattern

Bruno Silva, Patrícia Macedo

Sumário

- Introdução aos padrões de software (Software Patterns)
- Iterator Pattern
- Iterator Pattern aplicado às coleções
- Treino para casa

Padrões de Software | Introdução

- Em Engenharia de Software, um padrão é **uma solução geral** para um **problema** que ocorre com frequência dentro de um determinado contexto do **desenho de software**.
- Existem várias classificações para os padrões. Na UC de Programação Avançada vamos usar a seguinte classificação:
 - Padrões de Arquitetura (*Architectural Patterns*);
 - Padrões de Desenho (*Design Patterns*).

Padrões de Software | Introdução

<https://www.youtube.com/watch?v=l8CRI7fSw4g>

Padrões de Software | GoF Patterns

- **Padrões de Arquitetura** (*Architectural Patterns*)
 - Resolvem um problema típico da arquitetura de software. Definem formas de organizar os vários componentes de um sistema de software MVC, MVP, DAO.
- **Padrões de Desenho** (*Design Patterns*)
 - Resolvem problemas específicos e localizados (ex: como criar uma variável global, como adaptar o comportamento de uma classe, como criar uma família de classes etc.).

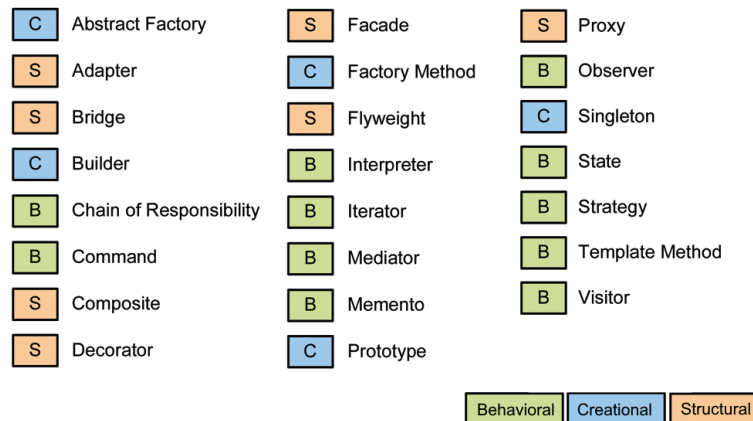
Padrões de Software | Vantagens

- Aprende-se com a experiência de outros;
- Utilizam-se soluções amplamente testadas;
- Permite utilizar uma linguagem comum entre os designers e programadores. Melhora-se assim a comunicação entre a equipa e a documentação dos sistemas;
- Leva ao uso de boas práticas no desenvolvimento de software orientado a objetos, obtendo-se assim software de melhor qualidade.

Padrões GoF

Design Patterns: Elements of Reusable Object-Oriented Software é um livro de engenharia de software escrito por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides que descreve 23 padrões clássicos de desenho. Os 4 autores do livro ficaram conhecidos por **GoF** (Gang of Four).

THE 23 GANG OF FOUR DESIGN PATTERNS



Especificação de um padrão de software

- Um padrão de desenho deverá ser especificado indicando:
 - O **nome** (cria um vocabulário de padrões);
 - O **problema** (diz quando aplicar o padrão);
 - A **solução** (descreve os elementos do design);
 - As **consequências** (de aplicar o padrão de desenho).

Iterator | Motivação

Para percorrer uma coleção do tipo *Set*, *Map*, *Tree*..., usa-se um ciclo *foreach*

```
Set<Integer> set = new HashSet();  
//add elements..,print elements  
for (Integer elem: set)  
    System.out.print(elem + " ");
```

```
Map<Integer,String> map = new HashMap();  
//add elements..  
//print keys  
for (Integer key: map.keys())  
    System.out.print(key + " ");
```

Os ciclos *foreach* só são possíveis de realizar se a coleção for **Iterable**.

- Uma coleção que implementa a interface **Iterable** é uma coleção que implementa o **padrão Iterator**.

Iterator | Iterator em JAVA

A interface `Iterable` e a interface `Iterator` são interfaces disponibilizadas pelo Java, pois as coleções do Java implementam esse padrão.

```
public interface Iterable<E>{  
    Iterator<E> iterator();  
}
```

```
public interface Iterator<E>{  
    boolean hasNext(); // devolve true se existir ainda um próximo elemento  
    E next();          // devolve o próximo elemento da sequência  
}
```

Iterator | Uso do padrão Iterator

Embora normalmente em JAVA usemos o ciclo foreach, para iterar os elementos de uma coleção, poderemos também usar diretamente os métodos disponibilizados pela interface Iterator.

```
Set<Integer> set = new HashSet();  
//add elements..  
//print elements  
for (Integer elem: set)  
    System.out.print(elem + " ");
```

```
Set<Integer> set = new HashSet();  
//add elements..  
//print elements  
Iterator<Integer> it=set.iterator();  
while(it.hasNext())  
    System.out.print(it.next() + " ");
```

Iterator | Exemplo de Aplicação

Problema Concreto:

Tomemos como exemplo a ADT *Stack* (que implementamos nas aulas iniciais) e que é uma coleção do tipo *Stack*. Queremos percorrer uma instância de *Stack* sequencialmente do topo para a base, sem destruir o conteúdo da variável **stack**.

```
Stack<Integer> stack = new StackArray();  
for (int i = 0; i < 20; i++)  
    stack.push(i);  
for (Integer i: stack)  
    System.out.print(i + " ");
```

Solução: Para percorrermos o stack usando um ciclo foreach, teremos que implementar o **padrão Iterator**.

Iterator | Exemplo de Aplicação

1. Tornar o tipo `Stack` `Iterable`
2. Implementar o método `iterator` nas classes que implementam a interface `Stack`.
3. Criar uma **inner classe** na implementação do `Stack` que implemente a interface `Iterator`.

Iterator | Exemplo de Aplicação

(1) Garantir que a **Stack** é *Iterable*

```
public interface Stack<E> extends Iterable<E>
```

(2) Implementar método **iterator**

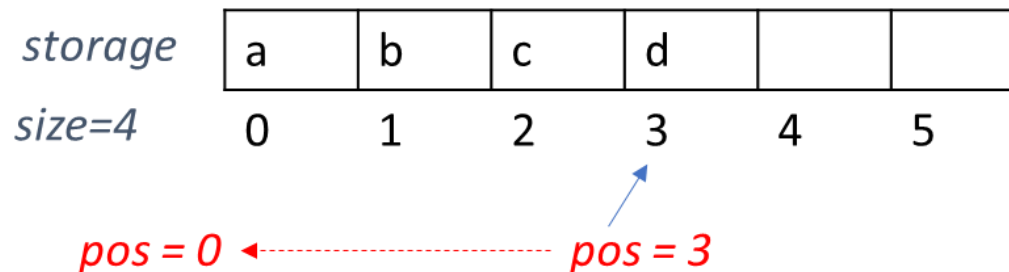
```
public class StackArray<E> implements Stack<E> {  
    private E[] storage;  
    private int size;  
    private final static int MAX = 500;  
  
    //other code from Stack Array  
    @Override  
    public Iterator<E> iterator() {  
        return new IteratorStack();  
    }  
  
    // continue...
```

Iterator | Exemplo de Aplicação

(3) Implementar a inner class `IteratorStack` que implementa a interface `Iterator` :

- Implementar o método `hasNext()` ;
- Implementar o método `next()` .

Nota: A implementação depende da estrutura de dados usada para suportar o stack. Neste caso usamos a classe *StackArray*.



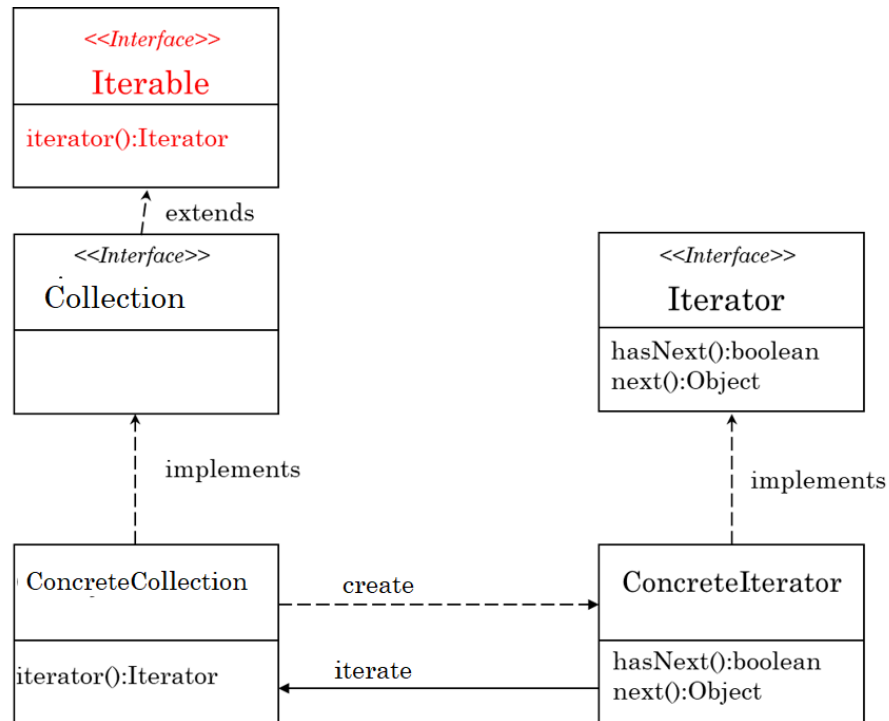
Iterator | Exemplo de Aplicação

(cont)

```
private class IteratorStack implements Iterator<E> {  
    private int pos;  
  
    public IteratorStack() {  
        pos=size-1;  
    }  
    @Override  
    public boolean hasNext() {  
        return pos>=0;  
    }  
    @Override  
    public E next() {  
        E elem= storage[pos];  
        pos--;  
        return elem;  
    }  
}
```


Iterator | Especificação

- Um padrão é uma solução padronizada para um problema comum.
- O padrão *Iterator* cria uma forma padronizada de iterar sequencialmente os elementos de um contendor.



Iterator | Especificação

PARTICIPANTES

- **Iterator**
Define uma interface para aceder e percorrer os elementos.
- **Concreteliterator**
Implementa a interface *Iterator*.
Mantém a informação da posição atual de iteração.
- **Coleção**
Define uma interface para criar um objeto *Iterator*.
- **Coleção Concreta**
Implementa a interface que cria o *Iterator* para retornar o *IteradorConcreto* apropriado.

Iterator | How To in Java

1. Na colecção em que pretende implementar o iterador, implemente o método `Iterator<E> iterator()` , que devolve uma instância do iterador da classe `ConcreteIterator` ;
2. Implemente a classe interna privada `ConcreteIterator`
`private class ConcreteIterator implements Iterator<E> ;`
3. Implemente os métodos `next()` e `hasNext()` , na classe `ConcreteIterator` .

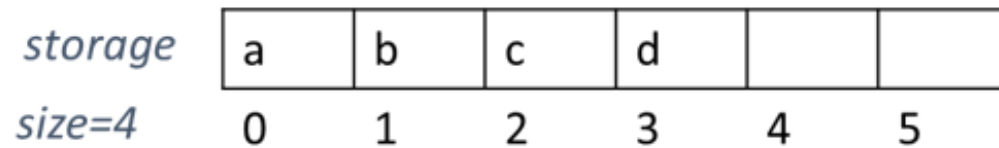
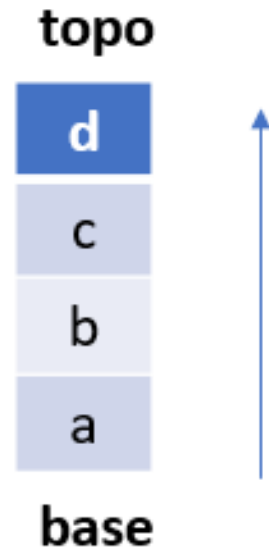
Iterator | Mais um exemplo (1)

Objetivo Implementar um iterador adicional na classe StackArray que itera os elementos da **base** para o **topo**.

- Como não é possível ter dois métodos denominados `iterator`, vamos criar o novo método com o nome `iteratorBaseToTop`.

```
@Override
    public Iterator<E> iteratorBaseToTop() {
        return new IteratorReverseStack();
    }
```

Iterator | Mais um exemplo (2)



pos=0 -----> pos=4 (size-1)

Iterator | Mais um exemplo (3)

```
private class IteratorReverseStack implements Iterator<E> {  
    private int pos;  
  
    public IteratorReverseStack() {  
        pos=0;  
    }  
    @Override  
    public boolean hasNext() {  
        return pos<size;  
    }  
    @Override  
    public E next() {  
        E elem= storage[pos];  
        pos++;  
        return elem;  
    }  
}
```

Iterator | Mais um exemplo (4)

- Iterar do Topo -> Base e da Base ->Topo

```
Stack<Char> stack = new StackArray();  
//add elements..  
//print elements : topo->base  
Iterator<Char> it1=stack.iterator();  
while(it1.hasNext())  
    System.out.print(it1.next() + " ");  
  
//print elements : base->topo  
Iterator<Char> it2=stack.iteratorBaseToTop();  
while(it2.hasNext())  
    System.out.print(it2.next() + " ");
```

Iterator | Treino em casa

Faça Clone do projeto

https://github.com/estsetubal-pa/Iterator_Template.git

E realize os exercícios especificados no README

Iterator | Referências

Web

- <https://refactoring.guru/design-patterns/iterator>
- https://www.tutorialspoint.com/design_pattern/iterator_pattern

Videos

- <https://www.youtube.com/watch?v=wqD4fOiGep4>
- <https://www.youtube.com/watch?v=QCWJWfuAfJc>